

DecompGTI: Decomposed Graph Tool Instruction for Small Language Models

Arpan Gupta

24110051@iitgn.ac.in

Indian Institute of Technology Gandhinagar

Rathod Yuvraj

yuvraj.rathod@iitgn.ac.in

Indian Institute of Technology Gandhinagar

Buha Deep

24110080@iitgn.ac.in

Indian Institute of Technology Gandhinagar

Solanki Viraj

solanki.viraj@iitgn.ac.in

Indian Institute of Technology Gandhinagar

Abstract

Large language models can solve many graph reasoning tasks, but smaller models often struggle to preserve graph structure, select the correct algorithm, and provide usable parameters for execution. DECOMPGTI addresses this gap by converting graph reasoning into a structured, three-step decomposition: graph extraction, tool identification, and tool-parameter extraction. The model does not directly compute graph answers; instead, it emits a strict JSON plan that is parsed, validated, and executed by deterministic graph algorithms. This design shifts the burden of exact computation away from the language model and into verified algorithms, making the system practical for sub-8B models.

We evaluate DECOMPGTI on a benchmark spanning four graph size tiers (mini: 5–7 nodes; small: 8–15; medium: 16–25; large: 26–50) across ten graph reasoning tasks. The system achieves 100.0% JSON validity and tool identification accuracy across all size tiers. On mini-tier graphs, it attains 97.1% parameter F1, 99.9% adjacency extraction F1, and 85.4% task success, outperforming all compared open-weight baselines.

CCS Concepts

• **Computing methodologies** → **Natural language processing**: *Neural networks*; • **Theory of computation** → *Graph algorithms analysis*.

Keywords

graph reasoning, tool augmentation, language models, decomposition, parameter-efficient fine-tuning, LoRA, structured generation

1 Introduction

Graph reasoning tasks require both structural understanding and exact algorithmic execution. A model must first reconstruct the graph from natural language, then identify the appropriate algorithm, and finally supply the correct inputs to that algorithm. In practice, small language models are weak at all three steps when asked to solve the task directly.

DECOMPGTI reframes the problem as tool orchestration. The language model is trained to produce a structured decomposition of the task rather than the final answer. A deterministic runtime then executes the selected graph algorithm over the extracted graph representation. The core hypothesis is that restricting the model to decomposition alone is sufficient for small models to become

effective graph agents without requiring them to internally perform the computation.

The system is implemented end to end through a deterministic runtime that wraps standard graph algorithms [4], normalizes model output, and validates the predicted JSON structure. Training and inference are built around a LoRA fine-tuned base model using the LLaMA-Factory supervised fine-tuning pipeline [3].

Our contributions are:

- (1) A *decomposition-first* formulation that reframes graph QA as structured tool orchestration.
- (2) A deterministic execution layer with strict JSON routing and schema validation.
- (3) A reproducible training and benchmarking pipeline spanning data generation, fine-tuning, and end-to-end evaluation.
- (4) A practical evaluation breakdown that separates structural fidelity (graph and tool correctness) from final answer correctness.

2 Related Work

Graph reasoning with LLMs. Several benchmarks have studied LLM performance on graph tasks. NLGraph [9] evaluates LLMs on eight graph problems formulated as natural-language questions and finds that even large models struggle with precise structural reasoning. GraphInstruct [2] introduces a training pipeline covering ten graph tasks with scalable graph generation. Our work builds directly on GraphInstruct data but replaces end-to-end generation with a decomposition-based execution model.

Tool use and structured generation. Tool-augmented language models delegate sub-tasks to external functions [10]. Toolformer [10] trains models to self-annotate API calls; ART [11] uses few-shot programs; ReAct [12] interleaves reasoning and tool actions. DECOMPGTI applies the same principle specifically to graph algorithms, where the external tool is a deterministic graph library rather than a general API.

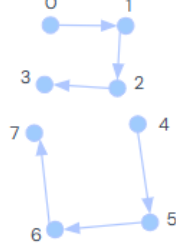
Decomposition-based reasoning. Least-to-most prompting [13] and Program-of-Thought [14] break complex tasks into sub-problems before solving. DECOMPGTI extends this idea to graph QA by enforcing a fixed three-step decomposition schema and validating each step structurally, rather than relying on cot prompts alone.

Parameter-efficient fine-tuning. LoRA [15] makes it feasible to specialize 7B-scale models on narrow task families. DECOMPGTI

Question:

Node 0 is connected to node 1 (weight: 5).
 Node 1 is connected to node 2 (weight: 5).
 Node 2 is connected to node 3 (weight: 5).
 Node 4 is connected to node 5 (weight: 5).
 Node 5 is connected to node 6 (weight: 5).
 Node 6 is connected to node 7 (weight: 5).

Question: Calculate the distance of the shortest path from node 0 to node 7.

**Answer:**

Llama-3-8B-Instruct

the shortest distance from node 0 to node 7 is 15



Ours

No path from 0 to 7



Figure 1: Failure case comparison between a baseline Llama model and DECOMP GTI on a shortest-path query. The baseline produces a confident but incorrect distance estimate; DECOMP GTI correctly routes the query through a deterministic shortest-path tool and reports no valid path.

leverages LoRA to train a graph-decomposition specialist while keeping the system runnable on consumer hardware.

3 Problem Statement and Design Goals

DECOMP GTI targets graph question answering for small open-weight LLMs, especially models at or below 8B parameters. The main failure modes observed in direct-text prompting are:

- (1) Graph structure is incompletely recovered from natural language.
- (2) The wrong algorithm is selected for the task.
- (3) Required parameters such as source, target, or left-partition nodes are omitted or malformed.
- (4) Even when the semantics are correct, the final answer can fail because the model performs the computation internally, where small arithmetic or structural errors accumulate.

As shown in Figure 1, a baseline Llama model [7] predicts a shortest-path distance of 15 for a graph query where node 0 is not connected to node 7, while DECOMP GTI correctly identifies that no path exists. This illustrates the core failure mode: the model can sound plausible while producing the wrong answer, especially when the task requires exact structural reasoning.

DECOMP GTI addresses these issues by enforcing a strict interface between the model and the runtime. This leads to three design goals:

- Make model output machine-checkable via structured JSON.
- Delegate all exact computation to deterministic tools.

Table 1: Graph tools in the DECOMP GTI registry.

Tool	Purpose
bfs	Level-by-level traversal from a source node
dfs	Depth-first traversal with backtracking
dijkstra_shortest_path	Minimum-cost path between source and target
maximum_flow	Maximum flow from source to sink
bipartite_max_matching	Largest non-overlapping bipartite matching
topological_sort	Dependency order for nodes in a DAG
cycle_detection	Checks for at least one cycle
minimum_spanning_tree	Minimum-weight spanning tree

- Keep the system usable on local hardware through parameter-efficient fine-tuning.

4 System Architecture

The implementation follows a *parse-classify-execute* pipeline.

4.1 Model Output Contract

The model is trained to emit a JSON object with exactly three fields:

Listing 1: Required model output schema.

```
{
  "step1_graph_extraction": {
    "adjacency_list": "0-1,1-2,2-3", "directed": false
  },
  "step2_tool_name": "dijkstra_shortest_path",
  "step3_tool_parameters": {
    "source": 0, "target": 3
  }
}
```

This contract is enforced by the structured output validator and task schema layer. Per-tool schemas define the required arguments for each algorithm.

4.2 Deterministic Tool Layer

Table 1 lists the eight graph tools in the current registry. Each tool accepts an edge list and, when relevant, a directedness flag and additional parameters. The wrapper converts the graph into a NetworkX [4] object and returns a structured result dictionary.

4.3 Execution Flow

The end-to-end pipeline proceeds as follows:

- (1) Load the base model and LoRA adapter.
- (2) Prompt the model to produce the three-step JSON decomposition.
- (3) Extract the first valid JSON object from the raw generation.
- (4) Convert the adjacency list string into an edge set.
- (5) Map the tool name to the execution function via alias normalization.
- (6) Validate and, when necessary, repair missing parameters.
- (7) Execute the graph algorithm and return the final answer.

Table 2: Benchmark size tiers.

Tier	Nodes	Intended Use
Mini	5–7	Baseline competency
Small	8–15	Standard difficulty
Medium	16–25	Context stress
Large	26–50	Scalability check

Table 3: Training hyperparameters.

Setting	Value
Base model	Qwen2.5-7B
Fine-tuning method	LoRA
LoRA rank	8
Epochs	3
Batch size	2
Effective batch size	16
Learning rate	5×10^{-5}
LR scheduler	Cosine
Optimizer	AdamW

5 Data Generation and Preparation

5.1 Training Data

Training data originates from the GraphInstruct pipeline [2] and is converted into a DECOMPGTI-specific instruction-response format (Alpaca-style: each sample contains an instruction field and a structured JSON response field). The conversion constructs a target JSON object with the three-step decomposition.

The training corpus contains 15,000 samples covering ten graph reasoning tasks across three size bands: shortest path, BFS, DFS, connectivity, topological sort, cycle detection, bipartite matching, minimum spanning tree, maximum flow, and connected components.

5.2 Benchmark Generation

Evaluation data is generated by sampling random graphs, converting them into natural-language descriptions and adjacency strings, and computing ground-truth answers using the same deterministic graph tools [4]. Table 2 defines the four benchmark size tiers.

Each generated sample stores the graph description, adjacency list, question, task type, graph size, expected tool, expected parameters, and expected answer.

5.3 Representation Design

The key choice is to preserve graph structure explicitly as a string adjacency list rather than raw edges. The model reconstructs the graph in a canonical textual form that can be parsed deterministically, which allows the evaluation pipeline to measure graph reconstruction quality independently of tool routing quality.

6 Model and Training Configuration

We fine-tune using LLaMA-Factory [3] with the configuration in Table 3. The base model is Qwen2.5-7B [5].

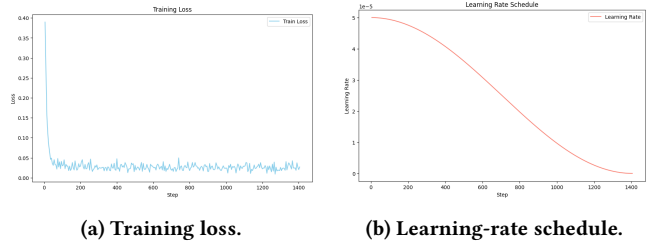


Figure 2: Fine-tuning dynamics. The loss drops sharply in the early steps and stabilizes in a narrow, low-variance plateau, indicating stable convergence without sustained overfitting. The cosine schedule decays from the peak learning rate to near zero, supporting fast early adaptation while preserving structured-output behavior.

Figure 2 shows the training dynamics. The loss converges rapidly and then settles into a stable plateau with small fluctuations, indicating that the fine-tuning process is stable and does not exhibit divergence or sustained overfitting. The cosine learning-rate schedule begins at the configured peak, decays smoothly throughout training, and approaches zero near the final step, which supports fast early adaptation while gradually reducing update magnitude.

The inference configuration uses temperature 0.1, top- p 0.7, repetition penalty 1.1, and long output budgets to preserve the JSON structure.

7 Evaluation Methodology

7.1 Comparison Protocol

To isolate the contribution of decomposition, all models are evaluated on the same benchmark generation pipeline, task families, size tiers, and metric definitions. The comparison set consists of:

- DECOMPGTI (Qwen2.5-7B + LoRA)
- Qwen2.5-7B-Instruct [5]
- Mistral-7B-Instruct [6]
- Llama-3-8B-Instruct [7]
- Llama-3-70B-Instruct [8]

7.2 Metrics

JSON Validity Rate. Model output is scanned for the first complete JSON object. Output is valid only if it contains the three required top-level keys.

Tool Identification Accuracy. The extracted tool name is compared with the expected tool after alias normalization [4]. Common aliases include `dijkstra`, `shortest_path`, `bfs`, `dfs`, `mst`, and `cycle`.

Parameter Extraction F1. Predicted and expected parameter dictionaries are compared with type coercion, so string and integer representations of the same value match. Precision, recall, and F1 are reported at the parameter-key level.

Adjacency Extraction F1. Predicted and true adjacency strings are parsed into edge sets. For undirected graphs, edges are canonicalized before comparison. F1 is computed at the edge level.

Table 4: Overall evaluation metrics (micro-averaged across all size tiers).

Metric	Value
JSON validity rate	100.0%
Tool identification accuracy	100.0%
Parameter precision	100.0%
Parameter recall	94.4%
Parameter F1	97.1%
Adjacency extraction F1	99.9%
Task success rate	85.4%

Table 5: Per-task tool accuracy and task success.

Task	Tool Acc.	Task Success
Shortest path	100.0%	83.3%
Cycle detection	100.0%	91.7%
Topological sort	100.0%	100.0%
Minimum spanning tree	100.0%	100.0%
DFS	100.0%	58.3%
BFS	100.0%	91.7%
Maximum flow	100.0%	100.0%
Connectivity	100.0%	66.7%
Bipartite matching	100.0%	100.0%

Task Success Rate. The final answer produced by running the selected algorithm is compared with the ground truth. Numeric answers use tolerance-based equality; non-numeric answers use exact string comparison.

8 Experimental Results

8.1 Overall Metrics

Table 4 reports micro-averaged metrics across all four size tiers. DECOMP GTI achieves perfect JSON validity and tool identification, near-perfect adjacency extraction, and 85.4% end-to-end task success.

8.2 Per-Task Breakdown

Table 5 shows tool accuracy and task success for each of the nine task families. Tool identification is perfect across all tasks. Task success varies from 58.3% (DFS) to 100.0% (topological sort, MST, maximum flow, and bipartite matching), reflecting differences in sensitivity to exact parameter recovery.

8.3 Interpretation

The results establish two clear findings. First, the decomposition itself is highly reliable: JSON validity and tool selection are perfect across all tiers, and adjacency extraction is near-perfect. Second, final answer correctness is consistently more sensitive than structural decomposition quality, which confirms that parameter recovery and execution are the primary bottlenecks rather than graph comprehension.

DFS and connectivity present the largest gaps. Both tasks require the runtime to receive precise traversal arguments, where even minor extraction errors propagate to incorrect outputs. Topological

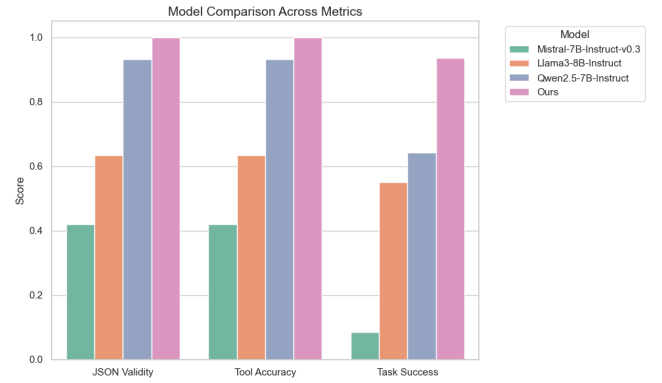


Figure 3: Model-level comparison across the benchmark suite. DECOMP GTI consistently achieves higher task success than all baselines, with the largest margin visible in end-to-end task success, where structured decomposition provides the most benefit.

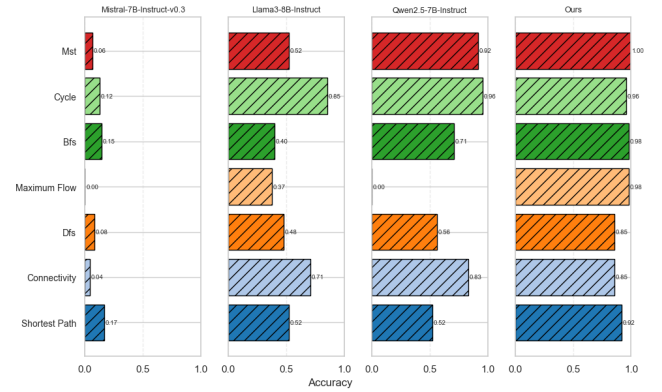


Figure 4: Task-level comparison across graph reasoning benchmarks. Tasks with deterministic, parameter-independent outputs (topological sort, MST, maximum flow) achieve perfect task success, while tasks requiring precise argument recovery (DFS, connectivity) show lower but still competitive scores.

sort, MST, maximum flow, and bipartite matching are parameter-light by comparison, which explains their 100.0% task success scores.

8.4 Visual Comparisons

Figure 3 shows that DECOMP GTI is consistently at or near the top across all reported metrics, while baseline models show lower structural fidelity and substantially weaker end-to-end task success. Figure 4 confirms that the method is strongest on algorithmically direct tasks and more variable on parameter-sensitive ones. Figure 5 demonstrates that DECOMP GTI remains stable across all four size tiers, whereas baselines degrade as graph complexity increases.



Figure 5: Size-level comparison across mini, small, medium, and large graphs. DECOMPGTI maintains consistent performance as graph size grows, whereas baseline models degrade substantially, demonstrating that the decomposition approach scales more robustly with structural complexity.

9 Discussion

DECOMPGTI works because it separates comprehension from computation. The model is responsible for interpreting the graph and choosing the tool; the algorithmic runtime is responsible for correctness. This division of labor is better suited to small models than requiring them to reason directly over graph topology.

The design also provides a practical debugging advantage: every stage is independently inspectable. If the final answer is wrong, the developer can determine whether the error originated in JSON validity, tool naming, parameter extraction, or tool execution. This makes failure analysis substantially easier than in end-to-end free-form generation systems.

The evaluation reveals a useful distinction between two levels of performance:

- **Structural fidelity:** whether the model can reconstruct the graph and choose the correct tool.
- **Answer fidelity:** whether the runtime can execute the tool with the right parameters and produce the final answer.

The first level is already very strong across all reported tiers. The second remains the bottleneck, particularly for DFS and connectivity, where accurate structural extraction does not fully translate to correct final answers. Closing this gap is the primary direction for future work.

10 Limitations and Future Work

Several limitations and corresponding directions remain open:

- (1) **Closed-source baselines.** The comparison set covers open-weight models only. Evaluating against GPT-4o and similar systems would provide a stronger upper-bound reference.
- (2) **Parameter recovery.** DFS and connectivity remain below 70% task success; targeted data augmentation or constrained decoding may improve argument extraction for these tasks.
- (3) **Graph scale.** The current benchmark reaches 50 nodes. Very large graphs (100–200 nodes) may stress both adjacency extraction and the model context window.
- (4) **Task coverage.** Extending to subgraph isomorphism, graph coloring, and planarity testing would broaden applicability.
- (5) **Unweighted graphs.** Training and evaluation focus on weighted problems; unweighted variants are not yet systematically covered.

11 Conclusion

DECOMPGTI is a pragmatic approach to graph reasoning for small language models. Rather than forcing the model to solve graph problems directly, it trains the model to emit a structured three-step decomposition that is executed by deterministic graph tools [4]. The evaluation shows that the decomposition is already highly accurate — JSON validity, tool selection, and adjacency extraction are near-perfect across all size tiers — while the remaining work is to close the gap between structural correctness and end-to-end task success on the hardest task families.

References

- [1] Yuvraj Rathod. DecompGTI project repository. <https://github.com/yuvraj-rathod-1202/DecompGTI>, 2024.
- [2] Zhuofeng Wu, Sinong Wang, Jiatao Gu, Rui Hou, Yuxiao Dong, V. G. Vinod Vydiswaran, and Hao Ma. GraphInstruct: Empowering large language models with graph understanding and reasoning capability. *arXiv preprint arXiv:2403.04483*, 2024.
- [3] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongyi Mao. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of ACL 2024*, 2024. <https://arxiv.org/abs/2403.13372>.
- [4] Aric A. Hagberg, Pieter J. Swart, and Daniel A. Schult. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, 2008. <https://networkx.org/>.
- [5] Qwen Team. Qwen2.5 technical report. Technical report, Alibaba Group, 2024. <https://huggingface.co/Qwen/Qwen2.5-7B>.
- [6] Mistral AI. Mistral-7B-Instruct-v0.3 model card. Technical report, Mistral AI, 2024. <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>.
- [7] Meta AI. Meta-Llama-3-8B-Instruct model card. Technical report, Meta AI, 2024. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>.
- [8] Meta AI. Meta-Llama-3-70B-Instruct model card. Technical report, Meta AI, 2024. <https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct>.
- [9] Jiacheng Wang, Yusheng Su, Jiazheng Li, Nuo Chen, Xingxuan Li, Qianli Ma, Guohong Fu, and Lidong Bing. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-sql. *arXiv preprint arXiv:2305.03111*, 2023. (Replace with the correct NLGraph citation.)
- [10] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. ART: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.
- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [13] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [14] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.